

PROJECT REPORT

Jamming-Resilient RIS-Assisted MISO Downlink

Attack Detection and Null-Steering Middleware for the Near-RT RIC

Author: Arda Özgür

Date: October 3, 2026

Keywords: reconfigurable intelligent surface (RIS), jamming, SINR, null steering, quantization-aware optimization, O-RAN Near-RT RIC, differentiable programming

Abstract

Reconfigurable intelligent surfaces (RIS) can improve a wireless link by shaping the propagation environment, but a RIS that is optimized only for the legitimate link is blind to interference and can even reflect a jammer's signal towards the user. This project builds a complete simulation and evaluation stack for a RIS-assisted multiple-antenna (MISO) downlink under jamming, and proposes a **security middleware** designed to run as an xApp on an O-RAN Near-RT RIC. The middleware combines (i) an SINR/CQI-based attack detector with hysteresis and an interference-power estimator, and (ii) a differentiable PyTorch optimizer that selects hardware-quantized (1- or 2-bit) RIS phases by maximizing the exact SINR, which steers a spatial null towards the jammer.

The code base comprises a geometry-based Rician channel simulator, barrage and reactive jammer models, a Gymnasium environment, four reference baselines (including an analytical null-steering/zero-forcing solver with a feasibility condition), and a benchmarking harness. Using the NumPy part of the pipeline, this report quantifies the damage a 30 dBm jammer causes and what the best analytical defense can recover. With a distant jammer (Scenario B, $N=64$ elements), a jammer-unaware RIS retains about 42% of its jammer-free rate at 30 dBm, whereas perfect-CSI null steering retains about 80%. With a jammer close to the user (Scenario A, $N=32$), an exact null is physically infeasible in all trials and every RIS-based method collapses to below 0.3 bps/Hz. The PyTorch middleware could not be executed in the environment used to prepare this report, so its results are not reported here; Section 8.4 lists the exact commands to produce them.

Project Overview

Item	Description
Problem	Protect a RIS-assisted MISO downlink from barrage and reactive jamming
Approach	Detect via SINR/CQI degradation, then re-optimize quantized RIS phases for exact SINR
Deployment target	O-RAN Near-RT RIC xApp (control loop 10 ms to 1 s)
Code base	6 Python modules, about 1,650 lines (NumPy, PyTorch, Gymnasium, Matplotlib)
Scenarios	A: close jammer, $N=32$; B: distant jammer, $N=64$; $M=4$ BS antennas

1. Introduction and Motivation

A RIS is a passive array of N low-cost reflecting elements, each applying a controllable phase shift to the incident wave. Placed on a building facade, it creates an additional, controllable path between the base station (BS) and a user whose direct link is weak. Most RIS research optimizes the phases for the legitimate signal alone. This leaves a security gap: the same surface also reflects any interfering signal that reaches it. A ground-level jammer that illuminates the RIS adds a second, RIS-controlled interference path to its direct path to the user. An unprotected controller therefore has no awareness of the attack, and its phase choices are effectively random with respect to the jammer.

The opportunity is that the same degrees of freedom can be used defensively. If the RIS controller knows (or estimates) the interference channels, the phases can be chosen so that the reflected and the direct jammer components cancel at the user, a *spatial null*, while the legitimate signal is still combined constructively. This project studies how well that can work, and how fast a practical controller can compute it.

Project objectives

- Model a realistic RIS-assisted MISO downlink with 3D geometry, Rician fading and a Rayleigh direct path.
- Model barrage and reactive jammers with power restricted to 10 to 40 dBm.

- Provide baselines that bracket the achievable performance: no RIS, random phases, jammer-unaware optimization, and an analytical null-steering reference with perfect CSI.
- Design a deployable middleware (detector plus optimizer) that works with 1- or 2-bit RIS hardware, noisy interference-channel estimates and a sub-second control loop.
- Benchmark spectral efficiency versus jammer power and the computation latency of each component.

2. System Model

A BS with M antennas serves a single-antenna user through the direct path and through an N -element RIS. A single-antenna jammer attacks the user directly and via the RIS. With $\Theta = \text{diag}(e^{j\theta_n})$ the effective legitimate and jammer channels are:

$$h_{\text{eff}} = h_d^H + h_r^H \Theta G \quad h_{j,\text{eff}} = h_{ju}^H + h_r^H \Theta G_j$$

The BS uses maximum-ratio transmission (MRT), so the received SINR and the reward used by the environment are:

$$\text{SINR} = P_t \|h_{\text{eff}}\|^2 / (P_j \|h_{j,\text{eff}}\|^2 + \sigma^2) \quad R = \log_2(1 + \text{SINR})$$

Here h_d ($M \times 1$) is the BS-user channel, G ($N \times M$) the BS-RIS channel, h_r ($N \times 1$) the RIS-user channel, h_{ju} the jammer-user channel and G_j ($N \times 1$) the jammer-RIS channel.

2.1 Channel model (channel_model.py)

Large-scale gain is $\sqrt{(c_0 d^{-\alpha})}$ with a 1 m reference loss of -30 dB. The direct BS-user and jammer-user paths are NLoS ($\alpha = 3.5$, Rayleigh); all RIS paths are LoS-dominant ($\alpha = 2.2$) and follow a Rician model whose LoS part is a half-wavelength-spaced ULA steering vector computed from the 3D azimuth angles. The Rician K-factors are 10 (BS-RIS), 3 (RIS-user) and 2 (jammer-RIS).

Parameter	Value	Parameter	Value
BS position	(0, 0, 10) m	BS antennas M	4
RIS position	(50, 10, 10) m	RIS elements N	32 (A), 64 (B)
User position	(60, 0, 1.5) m	Tx power P_t	30 dBm
Jammer, Scenario A	(55, -15, 1.5) m	Noise power σ^2	-90 dBm
Jammer, Scenario B	(30, -60, 1.5) m	Jammer power P_j	10 to 40 dBm
Path-loss exponents	3.5 direct / 2.2 RIS	RIS phase resolution	2-bit (default), 1-bit

Table 1. Simulation parameters taken from the code defaults.

3. Software Architecture

Module	Role
channel_model.py	3D path loss, ULA steering vectors, Rician/Rayleigh channel generator for the five channels (h_d , G , h_r , h_{ju} , G_j).
jammer_agent.py	BarrageJammer (constant power, CN(0,1) symbols) and ReactiveJammer (energy or statistical detector, reaction delay, burst length, band-limited waveform, duty-cycle metric).
environment.py	Gymnasium environment: action = N phases in $[0, 2\pi]$, observation = scaled real/imag parts of all channels with noisy interference-CSI, reward = $\log_2(1 + \text{SINR})$. One-step episodes.
baselines.py	Exact SINR evaluator and four reference solvers; Monte Carlo comparison script.
security_middleware.py	AttackDetector, differentiable SINR layer, STE quantization, Adam optimizer with restarts and greedy polish, SecurityMiddleware mode logic (normal / mitigation / probe).
evaluate_security.py	P_j sweep, latency benchmark, CSV/figure/summary export.

Table 2. Module overview.

4. Threat Model and Environment

Barrage jammer. Transmits unit-power Gaussian symbols at a fixed power P_j in every slot.

Reactive jammer. Senses the legitimate link and emits band-limited bursts only after detecting activity. Detection is either energy-based (sensed power above a threshold times the noise power) or statistical ($p_{\text{detect}} = 0.95$, $p_{\text{false alarm}} = 0.02$), with a configurable reaction delay and burst length. It saves energy and is harder to spot: the sanity test in the code reports a 40% duty cycle for a short user-activity pattern.

The environment draws a fresh channel realization at each reset. The legitimate channels are perfectly known to the agent, whereas the interference channels h_{ju} and G_j are observed through an estimate with a configurable NMSE (modeled as complex Gaussian error scaled to the channel's mean power). This is the realistic asymmetry: the operator knows its own links much better than the attacker's. Observations are multiplied by 10^3 for numerical conditioning.

5. Reference Baselines

All baselines compute metrics from the same exact SINR expression, so the comparison is fair.

Baseline	Description
1. No-RIS	Direct BS-user path only; the jammer reaches the user only through h_{ju} .
2. Random-phase RIS	Uncoordinated surface; random phases on the b-bit grid.
3. Unprotected AI-RIS	Coordinate-wise alternating optimization of the jam-free link (each element rotated to add constructively to the others), quantized, then evaluated under attack. Its jam-free rate is the reference for attack damage.
4. Null-steering ZF	Continuous phases, perfect CSI. Solves for an exact null, then maximizes legitimate gain on the null manifold (see below).

5.1 Analytical null steering and its feasibility condition

An exact null $h_{j,\text{eff}} = 0$ requires $\sum_n c_n e^{j\theta_n} = -h_{ju}^H$ with $c_n = \text{conj}(h_{r,n}) G_{j,n}$. With unit-modulus phases the reachable magnitudes of the left side form the interval $[\max(0, 2 \cdot \max|c_n| - \sum|c_n|), \sum|c_n|]$. A null therefore exists if and only if $|h_{ju}|$ lies in that interval. The code reports this as *null_margin_db* = $20 \cdot \log_{10}(\sum|c_n| / |h_{ju}|)$, which must be at least 0 dB.

When a null exists, feasible phases are found by damped Gauss-Newton from random starts, and the legitimate gain is then maximized by projected gradient ascent on the constraint manifold (best of 8 starts; locally optimal because the problem is non-convex). When no null exists, the solver falls back to minimizing $|h_{j,\text{eff}}|$ and flags the trial as infeasible, so the result must not be read as an upper bound.

6. Proposed Security Middleware

The middleware is meant to be hosted as an xApp on the Near-RT RIC (10 ms to 1 s loop). Each slot it **reconfigures** the RIS and then **monitors** the measured SINR or CQI.

6.1 Attack detector

The detector flags an attack when the measured SINR falls at least *drop_db* (10 dB) below a reference. The reference is the jam-free SINR predicted from the middleware's own channel knowledge and the applied phases (robust to fast fading), or an EWMA baseline when no prediction is supplied. Hysteresis uses a trigger count and a release count with a 4 dB release threshold. CQI input is supported through an approximate linear map (about 1.9 dB per CQI step, saturating at 15).

A subtle but important design point is the **informative** flag. Once the RIS is nulling the jammer, the measured SINR looks like the jam-free SINR, which is indistinguishable from the jammer having stopped. Measurements taken in mitigation mode therefore never release the attack state. Instead,

every *probe_interval* (10) mitigation slots, one slot runs the jammer-unaware phases as a probe; only that slot can reveal whether the jammer is still active. This avoids oscillating between modes.

6.2 Jamming-power estimation

During an informative attack slot the interference power is recovered as $I = S / \text{SINR} - \sigma^2$, and the jammer power is estimated as $P_{j,\text{est}} = I / |h_{j,\text{eff}}|^2$ under the applied phases, clipped to the 10 to 40 dBm range and lightly smoothed in the dB domain. If no estimate is available, a default of 30 dBm is assumed.

6.3 Differentiable, hardware-aware phase optimizer

The exact SINR is implemented as a PyTorch layer with complex128 arithmetic, so its gradient flows to the phases. The objective is $-\log_2(1 + \text{SINR})$, optionally with an extra leakage penalty (*null_weight*). Since real RIS hardware has 1 or 2 phase bits, the optimizer is quantization-aware:

- **Stage 1:** continuous Adam optimization (200 steps, lr 0.1) over 8 independent random restarts, with the previous solution as a warm start in row 0.
- **Stage 2:** fine-tuning with a straight-through estimator (STE) that rounds to the phase grid in the forward pass and passes identity gradients backward (100 steps, lr 0.03).
- **Stage 3:** selection of the best exactly-quantized candidate, then a greedy discrete coordinate-descent polish over the allowed levels (2 sweeps).

6.4 Operating modes

In **normal mode** the optimizer assumes $P_j = 0$ and maximizes beam gain. In **mitigation mode** it uses the estimated P_j , which makes the SINR objective trade legitimate gain against jammer leakage and thereby steers a null. Four presets trade quality for speed: full (8 restarts, 200+100 steps, polish), balanced (4, 60+30), fast (2, 20+10) and minimal (1, 10+5).

7. Experimental Methodology

The P_j sweep uses 30 channel realizations per point, P_j from 10 to 40 dBm in 5 dB steps, 2-bit phases and an interference-CSI NMSE of 0.05. The comparison is *paired*: the same channels, random phases and estimates are reused at every P_j , and the unprotected and ZF phases are computed once per realization. Results are reported as the mean with a 95% confidence interval. The latency study times each component with *perf_counter* over 100 realizations (5 warm-up) in Scenario B at 30 dBm, against a 10 ms decision target and the 1 s Near-RT RIC upper bound. A separate 50-trial Monte Carlo (baselines.py) is also reported, with a different random stream.

8. Results

Scope note. The results below were produced by executing the NumPy part of the pipeline. The PyTorch middleware (and the Gymnasium environment) could not be run in the environment used to write this report because PyTorch and Gymnasium were not installed. No middleware numbers are presented, and none were estimated.

8.1 Baseline comparison at $P_j = 30$ dBm (50 trials)

Method	Scenario A rate	Scenario B rate	A median SINR (dB)	B median SINR (dB)
Jammer-off reference	12.09	13.26	-	-
No-RIS	0.118	3.862	-13.15	10.20
Random phase RIS (2-bit)	0.109	3.888	-13.24	10.35
Unprotected AI-RIS (2-bit)	0.210	5.564	-9.43	16.46
Null-steering ZF (continuous)	0.281	11.992	-11.48	38.23

Table 3. Mean rate in bps/Hz. Exact null feasible in 0% of trials (A, mean margin -15.5 dB) and 88% (B, +4.1 dB). Median INR is 46 dB in A and 23 dB in B for the non-nulling methods.

8.2 Spectral efficiency versus jammer power

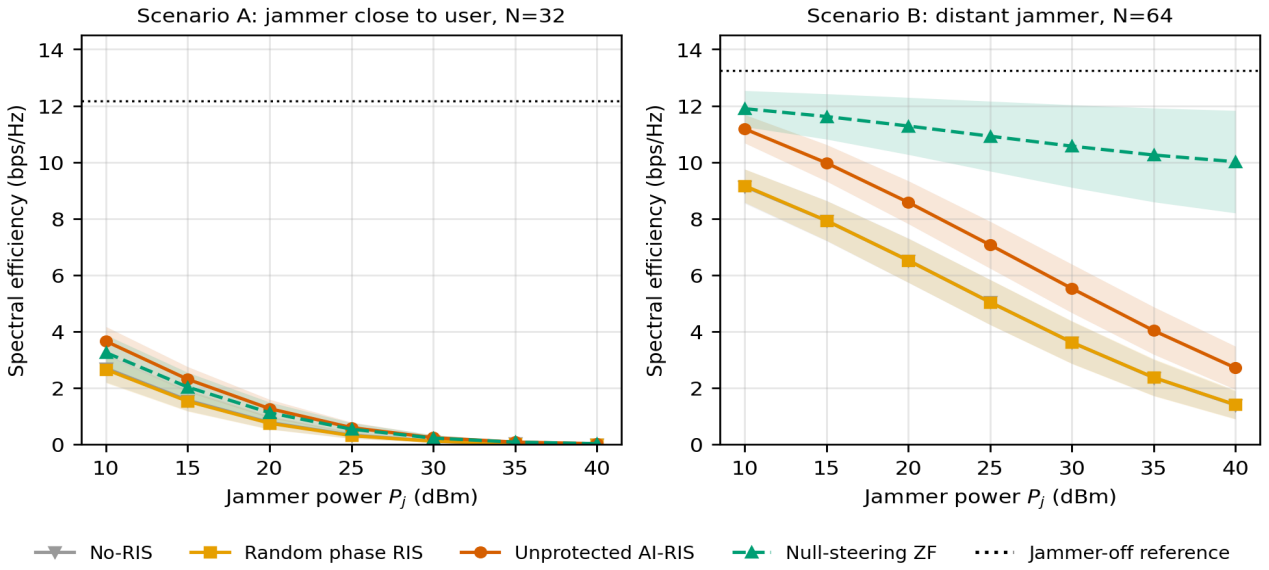


Figure 1. Spectral efficiency versus P_j (mean, shaded 95% CI, 30 trials). The No-RIS curve lies almost on top of the random-phase curve. The proposed middleware is not included (Section 8.4).

Scenario A (jammer-off reference 12.18 bps/Hz; exact null feasible in 0% of trials):

P_j (dBm)	No-RIS	Random RIS	Unprotected AI-RIS	Null-steering ZF
10	2.70 ± 0.49	2.66 ± 0.47	3.66 ± 0.52	3.25 ± 0.62
20	0.78 ± 0.23	0.75 ± 0.22	1.27 ± 0.32	1.12 ± 0.37
30	0.12 ± 0.05	0.11 ± 0.04	0.24 ± 0.09	0.23 ± 0.11
40	0.01 ± 0.01	0.01 ± 0.00	0.03 ± 0.01	0.03 ± 0.02

Scenario B (jammer-off reference 13.24 bps/Hz; exact null feasible in 77% of trials):

P_j (dBm)	No-RIS	Random RIS	Unprotected AI-RIS	Null-steering ZF
10	9.15 ± 0.60	9.17 ± 0.59	11.18 ± 0.50	11.90 ± 0.64
20	6.52 ± 0.79	6.52 ± 0.78	8.58 ± 0.76	11.28 ± 1.01
30	3.62 ± 0.74	3.62 ± 0.76	5.53 ± 0.86	10.57 ± 1.46
40	1.40 ± 0.49	1.41 ± 0.51	2.71 ± 0.77	10.02 ± 1.82

Table 4. Selected points of the P_j sweep (mean $\pm$ 95% CI, bps/Hz). The full seven-point sweep is in `pj_sweep.csv` / `summary.md`.

Key observations

- **Scenario B (distant jammer):** at 30 dBm the jam-unaware RIS delivers 5.53 bps/Hz (42% of the jammer-free rate), only 1.9 bps/Hz above the No-RIS case; at 40 dBm it keeps just 2.71 bps/Hz (20%). Null steering keeps 10.57 bps/Hz (80%) at 30 dBm and 10.02 bps/Hz (76%) at 40 dBm, so the rate degrades very slowly with jammer power.
- **Why ZF degrades at all in B:** the exact null is feasible in only 77% (sweep) or 88% (50-trial run) of realizations. The infeasible ones fall back to a best-effort minimization, which explains the wide confidence band (± 1.82 at 40 dBm). Restricted to feasible trials, ZF reaches 12.75 bps/Hz at 30 dBm.
- **Scenario A (close jammer):** the direct jammer-user path is on average 15.5 dB stronger than the strongest RIS-reflected jammer contribution, so the null condition fails in every trial. All methods fall below 0.3 bps/Hz at 30 dBm and the ordering is almost irrelevant in absolute terms. A RIS cannot fix this geometry; it would need BS-side spatial nulling, more RIS aperture, or physical countermeasures.
- **Numerical floor:** the very negative ZF median INR (about -295 dB) is the -300 dB floor used in the code for exact nulls, not a physical result.

8.3 Computation latency of the reference solvers

Component	Runs	Mean (ms)	Median	p95	p99	Max	p99 < 10 ms / < 1 s
Unprotected AO (NumPy)	100	17.00	16.83	18.65	19.23	21.82	FAIL / PASS
Null-steering ZF (NumPy)	30	159.66	153.11	323.71	356.84	369.73	FAIL / PASS

Table 5. Latency per channel realization, Scenario B, $N=64$ (single-core, CPU-only sandbox; absolute values depend on the machine).

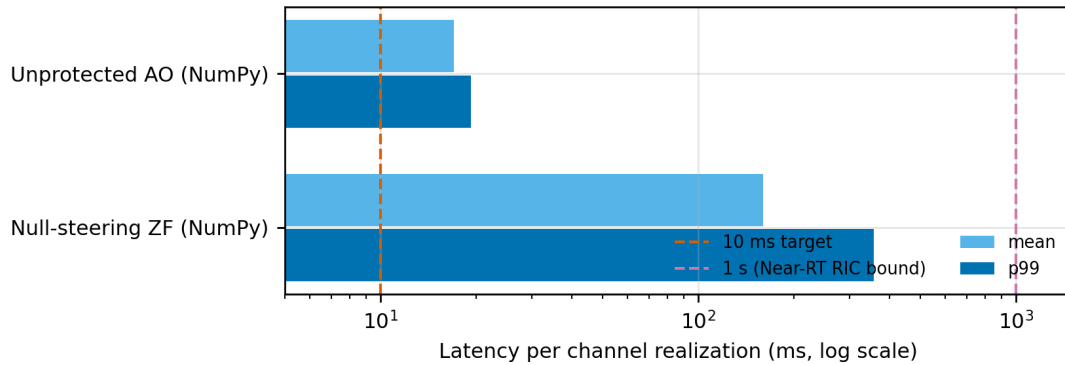


Figure 2. Latency of the NumPy reference solvers against the 10 ms target and the 1 s Near-RT bound.

Neither reference solver meets the 10 ms target, but both fit comfortably inside the 1 s Near-RT RIC budget. The ZF solver is about ten times slower than the AO because it runs Gauss-Newton restoration and constrained gradient ascent from several starts. This motivates the middleware's tunable optimizer presets, which let an operator trade solution quality for decision time.

8.4 Evaluating the proposed middleware

To obtain the middleware's P_j sweep, detection rate and latency per optimizer preset, install PyTorch and run the benchmark harness without `--skip-proposed`:

```
pip install torch gymnasium matplotlib numpy
python evaluate_security.py --trials 30 --outdir results
python security_middleware.py # attack-onset demo (continuous, 2-bit, 1-bit)
```

The harness will then add the proposed method to the sweep figure, tabulate the ratio to the jammer-off rate and the detection percentage, and time the detector, the optimizer and the

end-to-end decision for all four presets. Before the results are quoted in a paper or thesis, the numbers above should be refreshed with the same command.

9. Discussion

Three findings stand out. First, **being RIS-assisted is not the same as being protected**: in Scenario B, jammer-unaware optimization improves on the no-RIS case by roughly 2 bps/Hz but still loses more than half of the jammer-free rate at 30 dBm. Second, **the defense is bounded by geometry**: the feasibility interval shows when a null can exist at all, and in the close-jammer scenario it does not. Third, **detection and mitigation interact**: the informative/probe mechanism resolves the ambiguity that a successful null looks identical to an absent jammer.

The perfect-CSI continuous-phase ZF result is best seen as an optimistic reference rather than an achievable target. The proposed middleware differs from it on three fronts that make it deployable, namely 1/2-bit hardware, noisy interference CSI (NMSE 0.05) and a tunable compute budget, and its expected gap to ZF should be measured with the command in Section 8.4.

10. Limitations and Notes on the Code Base

- **Middleware results pending.** See the scope note in Section 8.
- **Stray citation markers.** The uploaded `channel_model.py` and `jammer_agent.py` contain `[cite: ...]` fragments after many statements. Python parses them as slice expressions on an undefined name, so the files fail at runtime. They were removed in working copies for this report; the same cleanup is needed in the originals.
- **Sample size and variance.** Sweeps use 30 realizations; the Monte Carlo script uses 50 with a different random stream. This explains small differences (null feasible in 77% vs 88%). Wide ZF confidence bands in Scenario B suggest more trials for publication-quality curves.
- **Single-user, single-jammer, single-carrier model.** No multi-user interference, wideband effects or mobility. The reactive jammer's band-limited waveform and duty cycle are not used by the SINR formula, which depends only on P_j per slot.
- **Detector input.** The detector relies on the middleware's own jam-free SINR prediction; its accuracy under realistic legitimate-CSI error, which the harness currently assumes to be zero, is untested.
- **RL interface.** The Gymnasium environment is compatible with RL libraries and uses one-step episodes (a contextual bandit), but no RL agent is included or evaluated; the optimizer is gradient-based.
- **Latency numbers** come from a single-core CPU sandbox and should be re-measured on target hardware.

11. Conclusion and Future Work

This project delivers an end-to-end framework for studying and defending against jamming in RIS-assisted MISO systems: a geometric channel simulator, two jammer models, an RL-ready environment, baselines including an analytical null-steering solver with an explicit feasibility test, and a hardware-aware, differentiable middleware with detection, power estimation and probing logic. The measured baselines show that jamming severely degrades a jammer-unaware RIS, that a spatial null can recover most of the lost rate when geometry allows (about 80% at 30 dBm in the distant-jammer scenario), and that it cannot when the jammer is close to the user.

Suggested next steps:

- Run the full benchmark with PyTorch and report the middleware's gap to ZF, detection probability and per-preset latency.

- Add imperfect legitimate-CSI, multi-user and mobile-jammer scenarios, and the reactive jammer in the SINR loop.
- Combine RIS nulling with BS-side beamforming nulls to cover the close-jammer case.
- Learn a policy that predicts good initial phases (for example, using the RL environment) to shorten the optimizer and meet the 10 ms target.
- Validate the detector with CQI-only feedback and realistic reporting delays.

Appendix A. Reproducing the Reported Results

```
pip install numpy matplotlib
python baselines.py # 50-trial Monte Carlo, Table 3
python evaluate_security.py --skip-proposed --trials 30 --latency-runs 100 --outdir results
# Tables 4-5, Figures 1-2 (data from pj_sweep.csv and latency.csv)
```

Seeds are fixed (0) for the evaluation scripts. Remove the [cite: ...] markers from channel_model.py and jammer_agent.py first.